博客园
kb.cnblogs.com

好的架构是进化来的，不是设计来的

来源: OneAPM 发布时间: 2015-11-22 17:36 阅读: 9329 次 推荐: 41 原文链接 [收藏]

摘要：对很多创业公司而言，随着业务的增长，网站的流量也会经历不同的阶段。从十万流量到一百万流量，再从一百万流量跨越到一千万甚至上亿的流量，网站的架构需要经历哪些变化？我们一起听听 58 同城的技术委员会执行主席沈剑在 OneAPM 技术公开课上的回答（以下演讲整理）：

好的架构不是设计出来的而是演进出来的

对很多创业公司而言，在初期的时候，我们很难在初期就预估到流量十倍以后、百倍以后、一千倍以后网站的架构会变成什么样。当然，如果在最初的时期，就设计一个千万级并发的流量架构，那样的话，成本是也是非常之高的，估计很难有公司会这样做。

所以，我们主要来讲架构是如何进行演化的。我们在每个阶段，找到对应该阶段网站架构所面临的问题，然后在不断解决这些问题的过程中，整个战略的架构就是在不断的演进了。

其实，在 58 同城建立之初，站点的流量非常小，可能也就是是十万级别，这也就意味着，平均每秒钟也就是几次的访问。此时网站架构的特点：请求量是比较低，数据量比较小，代码量也比较小。可能找几个工程师，很容易就做一个这样的站点，根本没什么「架构」可言。

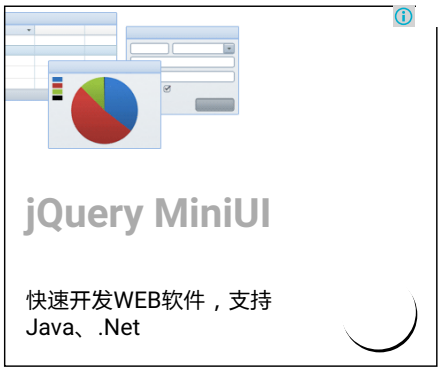
其实，这也是很多创业公司初期面临的问题，最开始58同城的站点架构用一个词概括就是「ALL IN ONE」，如下图所示：

就像一个单机系统，所有的东西都部署在一台机器上，包括站点、数据库、文件等等。而工程师每天的核心工作就是 CURD，前端传过来一些数据，然后业务逻辑层拼装成一些 CURD 访问数据库，数据库返回数据，数据拼装成页面，最终返回到浏览器。相信很多创业团队，初期做的工作也是类似，每天写代码，写 SQL、接口参数、访问数据等等。

这里需要说明一个问题，大家都知道目前 58 同城使用的是 Windows、IIS、SQL-Sever、C# 这条路。现在很多创业公司可能就不会这么做。58 同城为什么当时选择了这条路？原因是公司招聘的第一个工程师和第二个工程师只会这个，所以只能走这条路。

如果可以重来？那么会选择LAMP

很多创业的同学可能会想，如果我们初期希望做一个产品的话，我们应该使用什么架构？ 如果让我们重来，



搜索文章

推荐链接
程序员找工作，就在博客园招聘频道
程序员问答平台，解决您的技术难题
知识库iPad App，期待您的体验

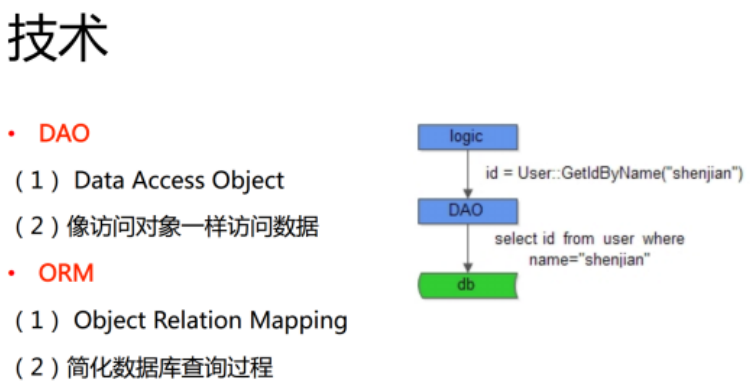
软件设计热门文章
UML用例图总结
深度理解依赖注入
理解RESTful架构
一步步构建大型网站架构
几种开源工作流引擎的简单比较

软件设计最新文章
从涂鸦到发布——理解API的设计过程
好的架构是进化来的，不是设计来的
再谈设计和编码
软件设计杂谈
RESTful架构详解

最新新闻
小岛秀夫新作PC版确认！PS4只是限时独占
粉丝福利Apple Music推出《星战》音乐专区
《史蒂夫·乔布斯》将推出数字/DVD/蓝光版
不仅是娱乐！虚拟现实能让人关注社会问题
2016年MacBook笔记本更新的那些事

热门新闻
腾讯互娱员工猝死，同事呼吁公司少加班
高三女孩被哈佛录取：只要一想到能编程，我就会不由
《神秘的程序员们》漫画26~28：《万年坑系列》I、
如果你写了25年代码，你将会遇到这些
怒赞！你知道为何你找不到软件工程师吗？

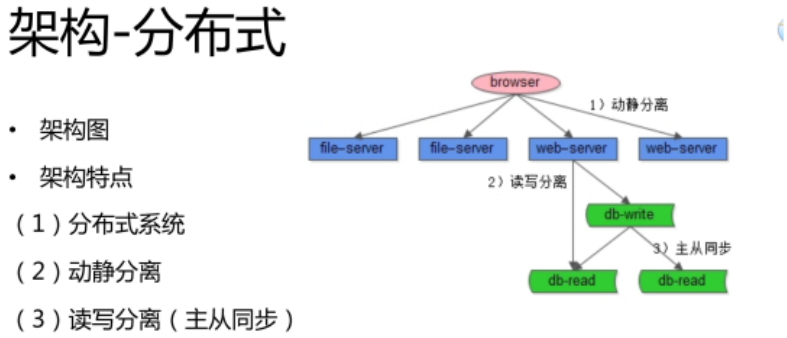
可能我们现在会选 LAMP，为什么？首先是无须编译，而且快速发布功能强大，从前端到后端、数据库访问、业务逻辑处理等等全部可以搞定，最重要的是因为开源产品，是完全免费的。如果使用 LAMP 搭建一个论坛，两天的时间就很足够了。所以，如果在创业初期，就尽量不要再使用 Windows 的技术体系了。



在这个阶段 58 同城面临的主要问题是什么？其实就是招人。很多工程师可能都是在培训学校里培训了3月就过来上班，所以他们写 CURD 的话很容易出错。当时，我们引进了 DAO 和 ORM。虽然那些培训了几个月的工程师可能写 CURD 不是特别的擅长，但是他们写面向对象的一些程序引入了 DAO 和 ORM，让他们不再直接面对 CURD 语句，这样就会相对容易一些。因为工程师比较擅长的是面向对象的数据，不是 CURD，所以我们当时引入了 ORM，总的来说，如果大家现在的项目处于一个初期孵化的阶段，DAO 和 ORM 能够极大的提高效率，而且可以降低出错的概率。

中等规模：流量跨过十万的阶段，数据库成为瓶颈

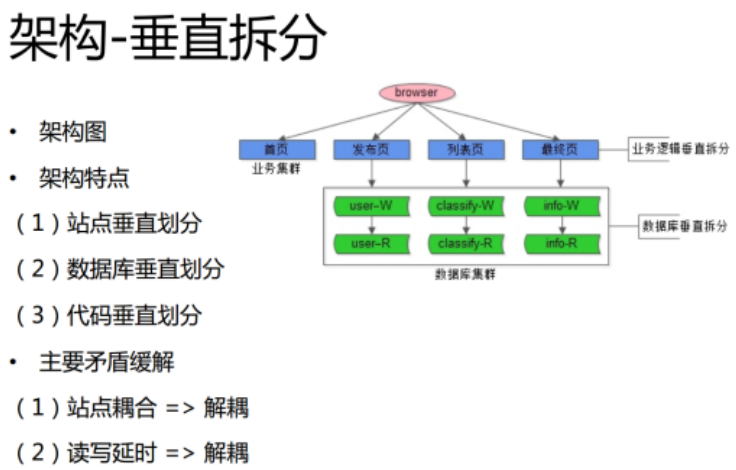
随着 58 同城的高速增长，我们很快跨越了十万流量的阶段。主要需求是什么？网站能够正常访问，当然速度更快点就好了。而此时系统面临问题包括：在流量的高峰期容易宕机，因为大量的请求会压到数据库上，所以数据库成为新的瓶颈，而且人多的时候，访问速度会很慢。这时，我们的机器数量也从一台变成了多台。现在的架构就采用了分布式，如下图所示：



首先，我们使用了一些非常常见的技术，一方面是动静分离，动态的页面通过 Web Server 访问，静态的像图片等就单独放到了一些服务器上。另外一点就是读写分离。其实，对 58 同城或者说绝大部分的站点而言，一般来说都是读多写少。对 58 同城来说，绝大部分用户是访问信息，只有很少的用户过来发贴。那么如何扩展整个站点架构的读请求呢？常用的是主从同步，读写分离。我们原来只有一个数据库，现在使用多个不同的数据库提供服务，这样的话，就扩展了读写，很快就解决了中等规模下数据访问的问题。

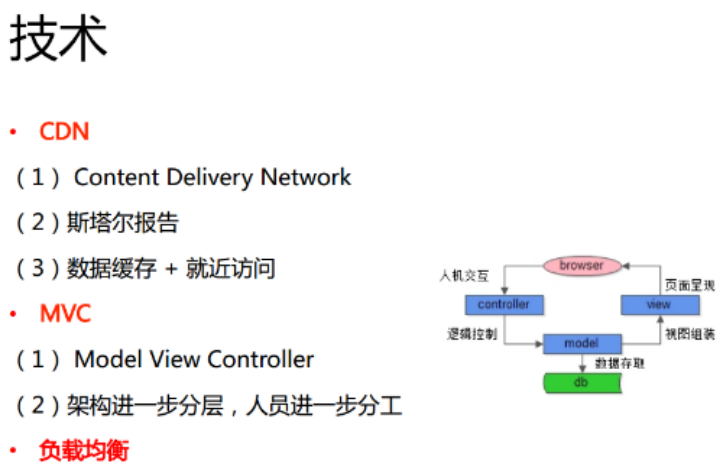
在这个阶段，系统的主要矛盾就是「站点耦合+读写延时」，58 同城是如何进行解耦，如何缓解延时呢？

对 58 同城而言，典型业务场景是主页，发布信息有发布页，信息聚合、标题聚合有列表页，点开一个标题有详细页，而这些站点都是耦合在一个程序中的，或者说耦合在一个站点中的，当我们有一个站点出现问题的时候，整个站点就会因为耦合一起出现问题。



第二个问题，大家都知道做数据库读请求和写请求，分布在不同的数据库上，这个时候如果再读取可能读到的是旧数据，因为读写有一个延时。如果有用户发帖子，马上去找的话肯定找不到，很可能带来的后果就是陆续在发布两条信息，这就是一个很大的问题。尤其是在请求量越来越大的时候，这个问题就更加突出。

在解决这些问题是，最先想到的是针对原来站点的核心业务做切分，然后工程师根据自己的站点和业务场景进行细分。首先，业务拆分是 58 同城最先尝试的优化。我们将业务垂直拆分成了首页和发布页。另外，在数据库层面，我们也随之进行了拆分，将大数据量拆分成一个个小的数据量。这样，读写延时就马上得到了缓解。尤其是在代码拆分成了不同的层面之后，站点耦合也得到了缓解，数据量加载速度也提升了很多。



当时，还使用了一些技术，前面也提到了对动态资源和静态资源进行拆分。其中，我们对静态资源使用了 CDN 服务，便于数据缓存和就近访问，访问速度得到很明显的提升。除此之外，我们还使用了 MVC 模式，擅长前端的去做展示层，擅长协作逻辑的工程师就做 Controller，擅长数据的人就负责数据，效率就会逐步的提高，最后就是负载均衡技术。

大流量：将整个 Windows 技术体系转向了 Java 体系

流量越来越大，当流量超过一千多万时，58 同城面对最大的问题就是性能和成本。此前，我提到58同城最初的技术选型是 Windows，应该是在 2006 年的时候，整个网站的性能变得非常之低。即使进行了业务拆分和一些优化，但是依然解决不了这个问题，所以我们当时做了一个非常艰难的决定，就是转型：将整个 Windows 技术体系转向了 Java 体系，这涵盖了操作系统、数据库等多个维度。

性能？成本？

- 脱胎换骨的转型！
- 增加性能，节省成本 => 全面转型开源技术

1）windows => Linux

2）sql-server => mysql

3）IIS => tomcat

4）C# => Java

5）...

其实，现在很多大的互联网公司在流量从小到大的过程中都经历过转型，包括京东、淘宝等等。对技术的要求越来越高，任何一个站点都不能挂，对站点的可用性要求也是越来越高。

就在这个时候，58同城业务量也出现一个爆发期。于是我们招聘了很多的工程师，大家一起写越来越多的站点，但是发现效率很低，经常做一些重复性的工作，比如参数解析等等。同时，业务之间相互依赖，无论是分类的子系统还是信息的子系统，二手车业务、房产业务都要访问用户和信息等一些底层数据，代码之间频繁的沟通，效率也不可能很高。

问题随之而来，站点数越来越多，数据量越来越大，机器数从最开始的几台上升到几百台的级别。那么如何提供整个架构的可用性呢？首先，在上层我们进行了一些改进和优化，再做进一步的垂直拆分，同时我们引入了Cache，如下图所示：

架构-高可用

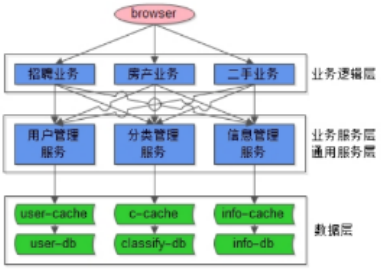
- 架构图
- 架构特点

（1）进一步垂直拆分

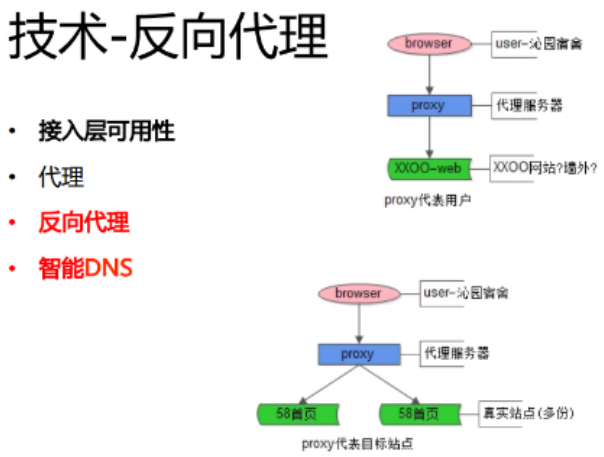
（2）分层抽象

（3）服务化

（4）水平拆分



在架构的改进上，我们构建了一个相对独立的服务层，这个服务层做的每个业务线都会写对应的代码。如果用户发出请求，就由这个服务层统一来管理，所有的上游业务线就像调用本地函数一样，通过 IDC 的框架来调用这个服务。整个用户登录先访问 Cache，如果 Cache 命中了就直接返回，如果 Cache 不命中，就会访问数据库，这样把数据库的数据拿到本地再放回 Cache，再打回上一轮。如此一来，业务逻辑全部封装在这个服务的上游管理，该业务逻辑只有服务层能够编写代码，然后由这个服务层集中管理、集中优化，这样就提高了效率。

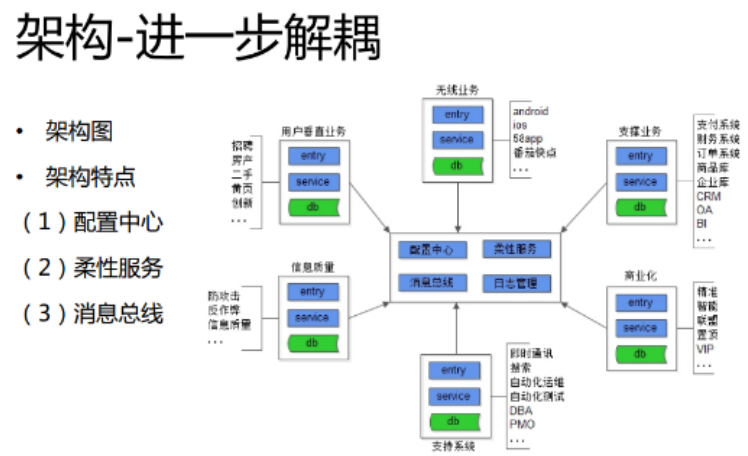


除此之外，为了保证站点的高可用，我们主要使用了反向代理技术。因为用户而言，他主要为了使用58同城的服务，他不关注访问是58同城或者有十台首页的服务器。58同城通过反向代理技术，通过 DNS 群，通过 LVS 技术，来保证接入层的高可用性，同时还保证了服务层、站点层、数据层的高可用。另外，为了保证高可用我们经常使用冗余的方法，无论是站点服务和数据服务都可以使用这种方式进行解决，一个站点不可用，我们就换一个站点，一个数据库不够用，我们就多加几个。当然，数据冗余也会带来一些副作用，如果数据量更新的话，那就需要将所有的“冗余”都要进行更新。

58同城也做了一个图片存储系统，开始都是存储在操作系统之上，随着新增站点、新增服务，压力就变得越来越大。于是，58同城就自建了站点框架和服务框架，现在这两个框架也已经开源（如何降低站点开发成本？https://github.com/58code/Argo 如何降低服务开发成本？ https://github.com/58code/Gaea ）只需要修改一些基本的配置就可以使用了。

当架构变成『蜘蛛网』，人肉已很难搞定！

随着用户量、数据量并发量进一步的增长，58同城也拓展了很多的新业务，那么对产品迭代速度要求就非常高，整体的架构对自动化的要求越来越高。



为了支撑业务的发展，技术团队对架构做了进一步的解耦，另外就是引入了配置中心，如果要访问任何一个服务，不会直接在本地的配置中留下一个服务，配置中心告诉这个服务的特点，如果扩展的话，配置中心自动下达消息，如果有机器要下线的话，配置中心会反向通过发邮件的方式进行通知。

而柔性服务是指当流量增加的时候，自动的新增服务。可以看到进一步解耦之后，有垂直业务、无线业务、集成业务等等，这些子系统之间都是通过配置中心相应之间发生关系的。

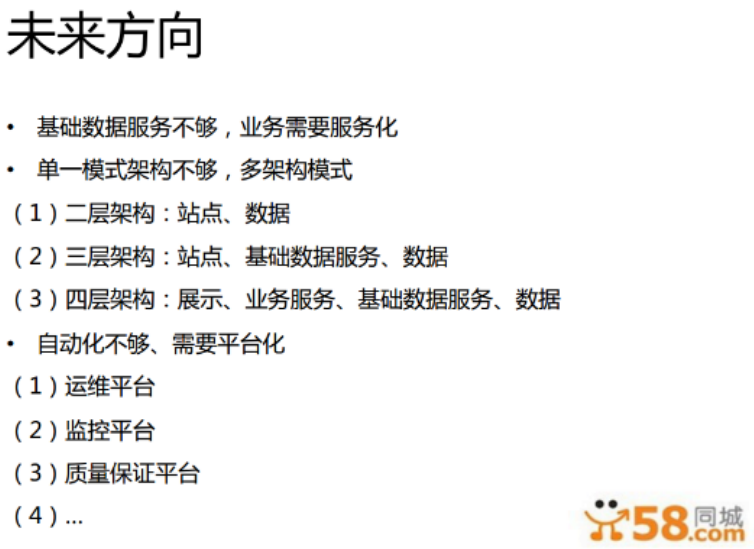
另一点就是关于数据库，当某一点成为一个业务线重点的时候，我们就会集中解决这个点的问题。最初期的时候每个业务线都要访问数据库，访问缓存，访问用户数据，于是我们把代码集中的放到了服务层。现在数据量越来越大，大家都要做数据切分，每个业务线都做切分，这个时候58同城的每个页面都面对这样的痛点，于是把这个痛点拿到集中的层面来解决。

最后一点就是效率矛盾，此时很多问题，靠「人肉」已经很难进行搞定了。这就需要自动化，包括回归、测试、运维、监控等等都要回归到自动化。

这里需要补充一点，就是在产品层面，我们引入了智能化，比如说智能推荐，主动推荐一些相关的话题；智能广告，通过一些智能的策略，让用户对广告的点击更多，增加对58同城的收入；智能搜索，在搜索的过程中加入一些搜索的策略，可以提高搜索的权重，也可以增加58同城的 PV。当然，所有的自动化的产品背后都是由技术在驱动。

未来的挑战

现在，58同城的流量已经突破的10亿的量级，那么架构上未来面临哪些挑战呢？一方面是无线化、移动化。另一方面就是需求的变化，我们必须加快迭代一些东西。如果拥有10亿的流量，却跑在一亿的架构上肯定是不行的。未来，我们会使用更多的并行计算、实时计算，如果能做到实时推荐，效果肯定非常好，这也是我们的挑战。最后一点，58同城现在的服务器大概在3000台左右，未来将拓展到1万台，这就是运维的挑战了。



总结：

最后做一个小的总结，网站在不同的阶段遇到的问题不一样，而解决这些问题使用的技术也不一样，流量小的时候，我们主要目的是提高开发效率，在早期要引入 ORM，DAO 这些技术。随着流量变大，使用动静分离、读写分离、主从同步、垂直拆分、CDN、MVC 等方式不断提升网站的稳定性。面对更大的流量时，通过垂直拆分、服务化、反向代理、开发框架（站点/服务）等等，不断提升高可用。在面对上亿级的更大流量时，通过中心化、柔性服务、消息总线、自动化（回归，测试，运维，监控）来迎接新的挑战。未来的就是继续实现移动化，大数据实时计算，平台化...

41 11

标签： 网站架构

« 上一篇：再谈设计和编码